

《移动终端软件开发技术》

Mobile Application Development

2. Android组件与生命周期

谢坤
计算机学院（国家示范性软件学院）

1 四大组件与生命周期

1.1 Android组件

■ Android组件 （调用的基本模块）

□ 组件

- 可以被调用的基本功能模块（程序内或程序间）
- 解决代码复用

□ 被调用组件的进程启动与否？

- 不启动组件所属的程序
- 允许共享组件直接实例化

1.1 Android组件

■ Android系统四大组件

□ Activity

- Android程序的呈现层，显示可视化的用户界面，并接收与用户交互所产生的界面事件
- Android应用程序可以包含一个或多个Activity，一般需要指定一个程序启动时显示的Activity

□ Service

- Service一般用于没有用户界面，但需要长时间在后台运行的应用
- 可公开Service的程序接口，供其他进程调用

1.1 Android组件

■ Android系统四大组件

□ BroadcastReceiver

- 用来接收广播消息的组件，不包含任何用户界面
- 启动Activity或者Notification以通知用户重要信息

□ ContentProvider

- 是Android系统提供的一种标准的数据共享机制，其他程序通过ContentProvider访问程序的私有数据
- Android系统内部提供一些内置的ContentProvider，为应用程序提供重要的数据信息

1.2 组件生命周期

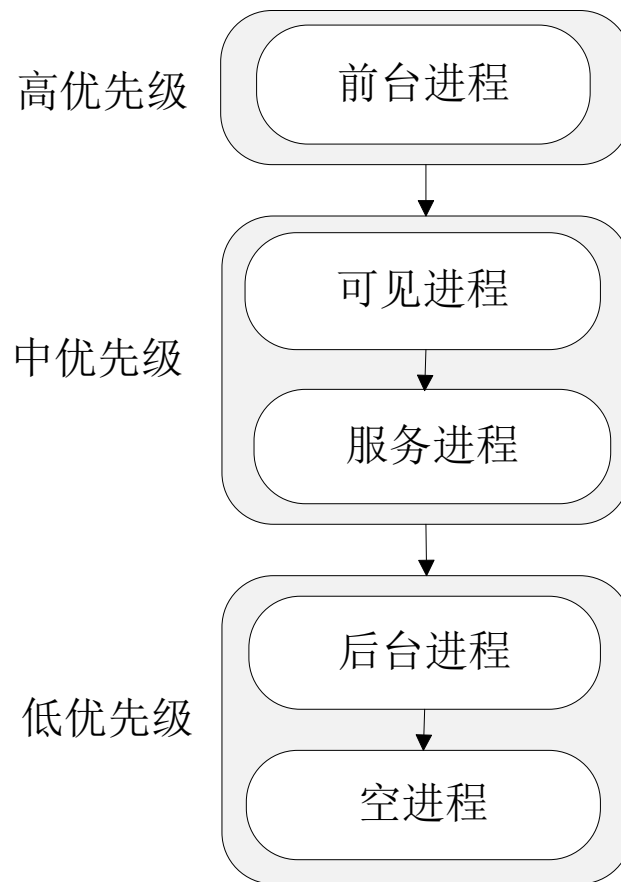
■ 组件生命周期

- 组件从建立到销毁的整个过程
- 组件在生命周期内状态的不断变化
 - 可见、不可见、活动、非活动等

1.3 程序生命周期

■ Android系统的进程优先级
从高到低分别为：

- 前台进程
- 可见进程
- 服务进程
- 后台进程
- 空进程



1.3 程序生命周期

■ 前台进程

- 用户正在交互的进程

■ 可见进程

- 程序界面能够被用户看见，却不在前台与用户交互，不响应界面事件的进程

■ 服务进程

- 包含已启动服务的进程

■ 后台进程

- 不包含任何已经启动的服务，而且没有任何用户可见的Activity的进程

■ 空进程

- 不包含任何活跃组件的进程

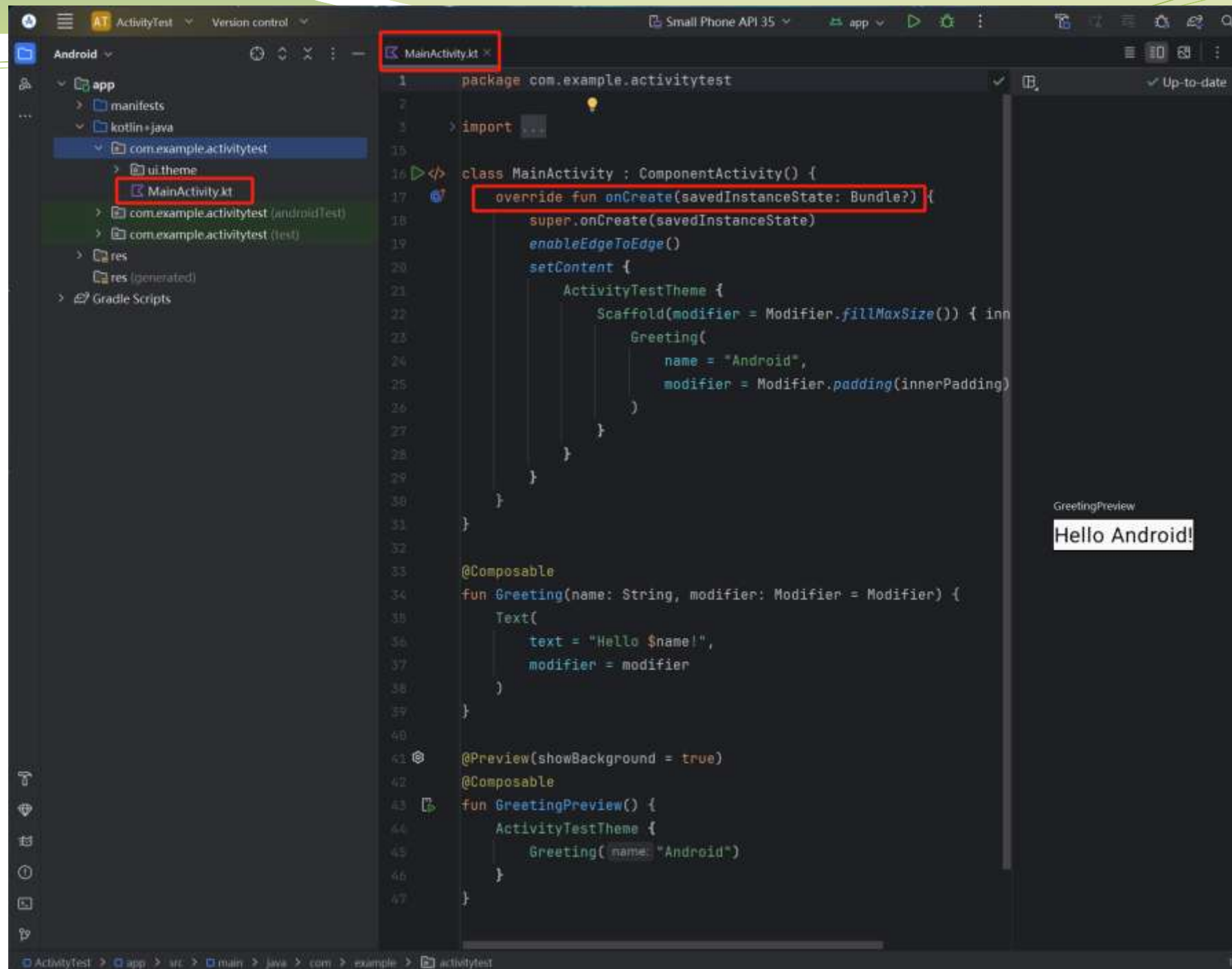
1.3 程序生命周期

■ 优先级的决定与变化规则

- 进程的优先级取决于所有组件中的优先级最高的部分
- 进程的优先级会根据与其他进程的依赖关系而变化

2 Activity的基本用法

2.1 创建项目



2.1 创建项目

```
16 class MainActivity : ComponentActivity() {  
17     override fun onCreate(savedInstanceState: Bundle?) {  
18         super.onCreate(savedInstanceState)  
19         enableEdgeToEdge()  
20         setContent {  
21             ActivityTestTheme {  
22                 Scaffold(modifier = Modifier.fillMaxSize()) { inn  
23                     Greeting(  
24                         name = "Android",  
25                         modifier = Modifier.padding(innerPadding)  
26                     )  
27                 }  
28             }  
29         }  
30     }  
31 }  
32  
33 @Composable  
34 fun Greeting(name: String, modifier: Modifier = Modifier) {  
35     Text(  
36         text = "Hello $name!",  
37         modifier = modifier  
38     )  
39 }
```

GreetingPreview

Hello Android!

2.2 Activity 注册

- Activity在AndroidManifest.xml中的注册:

com.example.activitytest.MainActivity

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ActivityTest"
        tools:targetApi="31">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.ActivityTest">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

2.3 添加按钮

```
<resources>
  <string name="app_name">ActivityTest</string>
  <string name="Button_1">Button 1</string>
</resources>
```

```
> import ...

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ActivityTestTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->
                    Greeting(
                        name = "Android",
                        modifier = Modifier
                            .padding(innerPadding)
                            .fillMaxSize()
                            .wrapContentSize(Alignment.Center)
                    )
                }
            }
        }
    }
}

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = { /*TODO*/ }) {
            Text(stringResource(R.string.Button_1))
        }
    }
}
```

GreetingPreview

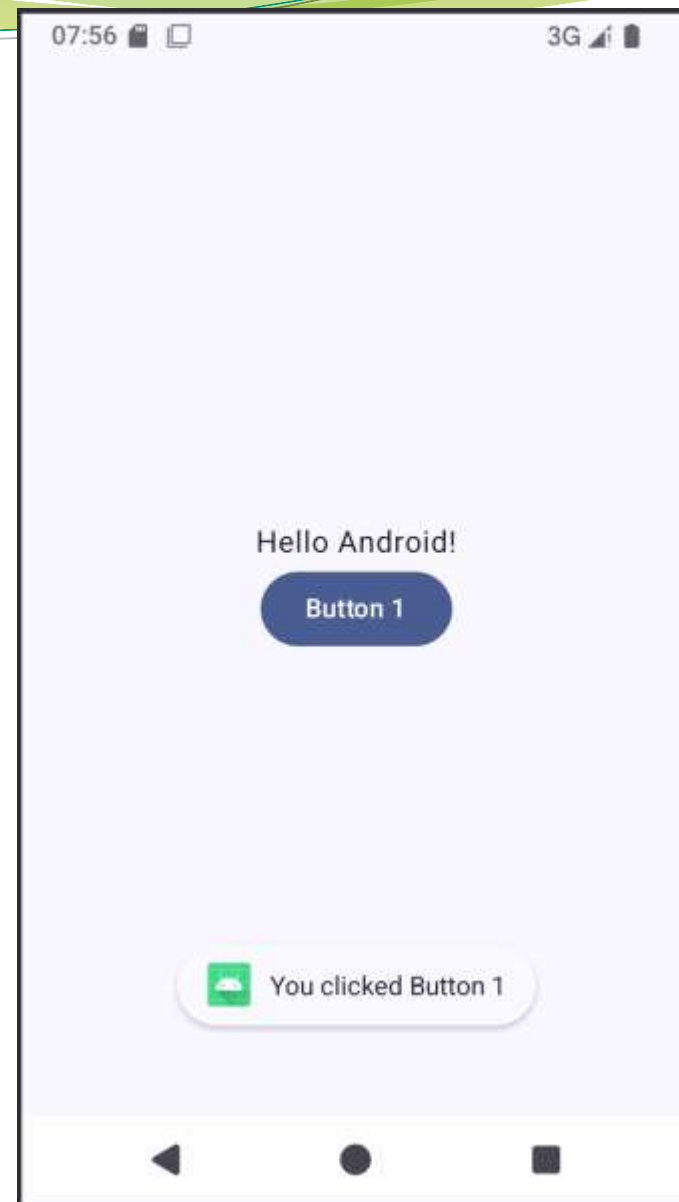
Hello Android!

Button 1

2.4 在Activity中使用Toast

■ Toast: 一种提醒方式

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = {
            Toast.makeText(context, text: "You clicked Button 1", Toast.LENGTH_SHORT).show()
        }) {
            Text(stringResource("Button 1"))
        }
    }
}
```



2.5 销毁Activity

- 按手机Back键;
- 调用finish()方法。

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = {
            activity?.finish()
        }) {
            Text(stringResource(R.string.Button_1))
        }
    }
}
```

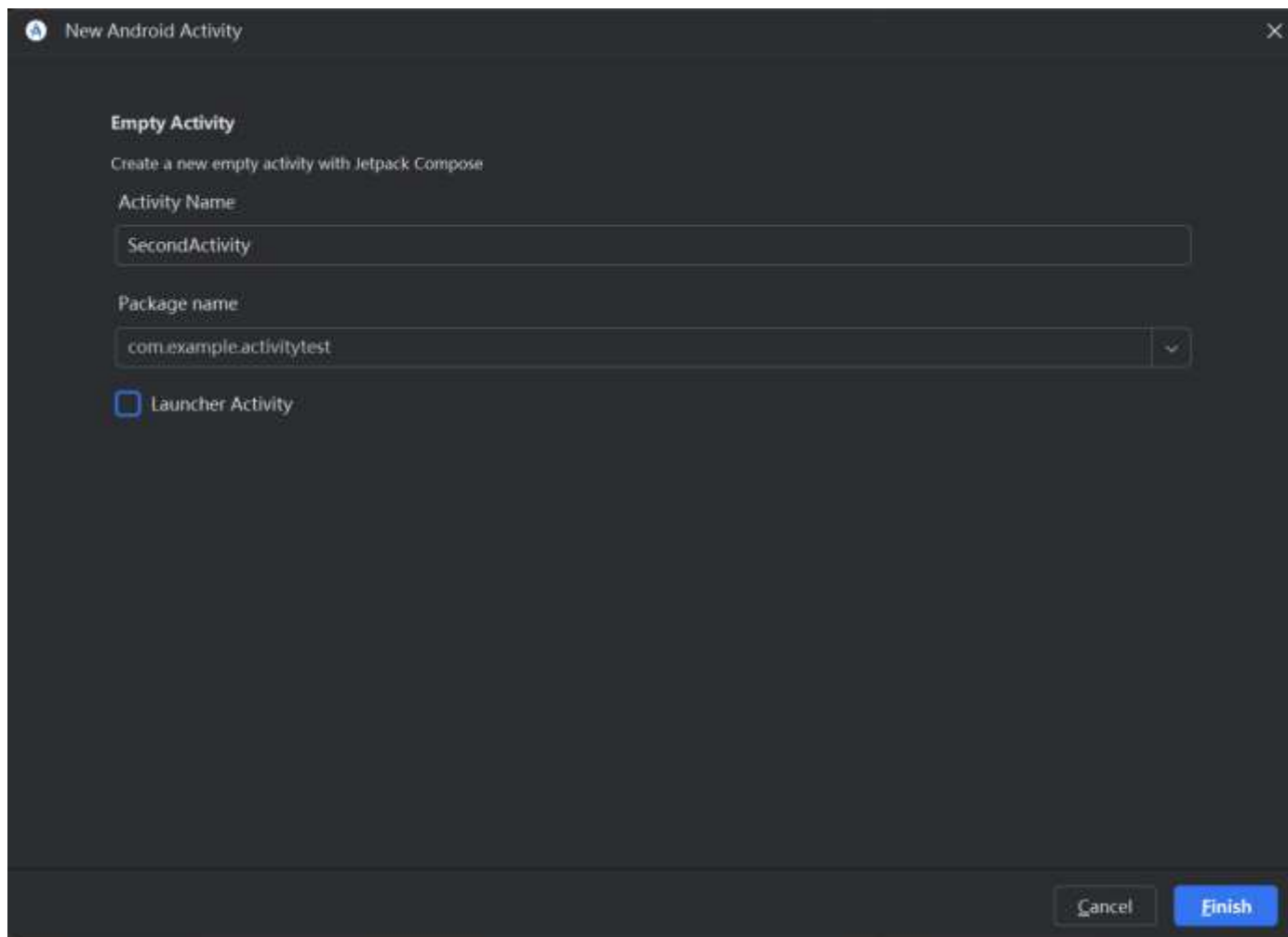

3 Intent的用法

3.1 什么是Intent

■ Intent是一种轻量级的消息传递机制

- 是一个动作的完整描述，包含了动作的产生组件、接收组件和传递的数据信息
- 用于组件之间数据交换
 - Activity、Service和BroadcastReceiver的数据交互
 - 启动Activity和Service
- 发送广播消息
 - 应用程序广播消息
 - 系统广播消息
 - 手机的信号变化
 - 电池的电量过低

3.2 创建另一个Activity



The screenshot shows the 'New Android Activity' dialog box in Android Studio. The dialog has a dark theme and a title bar with an Android icon and the text 'New Android Activity'. Inside the dialog, the section 'Empty Activity' is selected, with the subtitle 'Create a new empty activity with Jetpack Compose'. Below this, there are two input fields: 'Activity Name' with the text 'SecondActivity' and 'Package name' with the text 'com.example.activitytest'. At the bottom left, there is a checkbox labeled 'Launcher Activity' which is currently unchecked. At the bottom right, there are two buttons: 'Cancel' and 'Finish'.

New Android Activity

Empty Activity
Create a new empty activity with Jetpack Compose

Activity Name
SecondActivity

Package name
com.example.activitytest

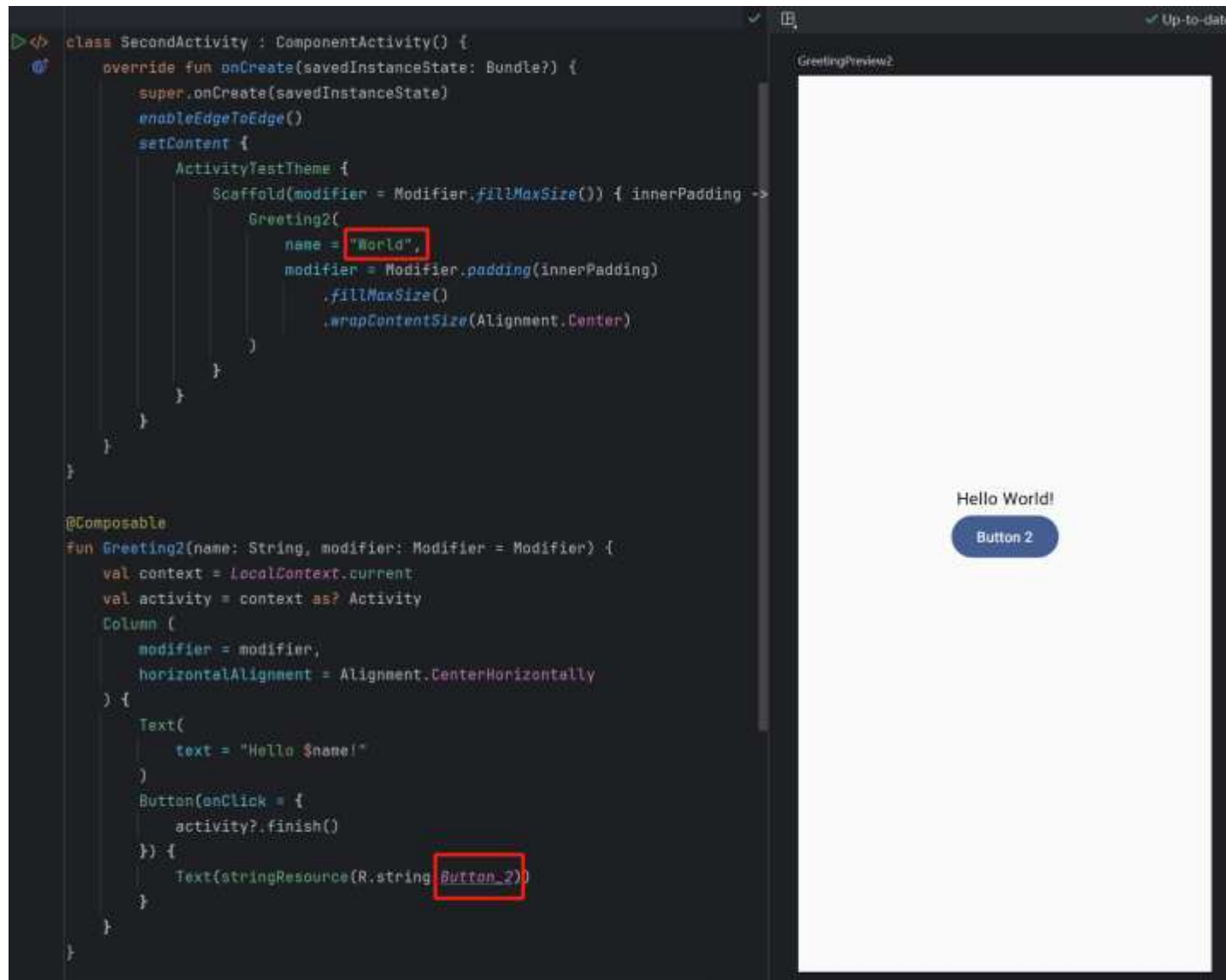
☐ Launcher Activity

Cancel Finish

3.2 创建另一个Activity

```
<resources>
    <string name="app_name">ActivityTest</string>
    <string name="Button_1">Button 1</string>
    <string name="Button_2">Button 2</string>
    <string name="title_activity_second">SecondActivity</string>
</resources>
```

```
<activity
    android:name=".SecondActivity"
    android:exported="false"
    android:label="@string/title_activity_second"
    android:theme="@style/Theme.ActivityTest" />
```

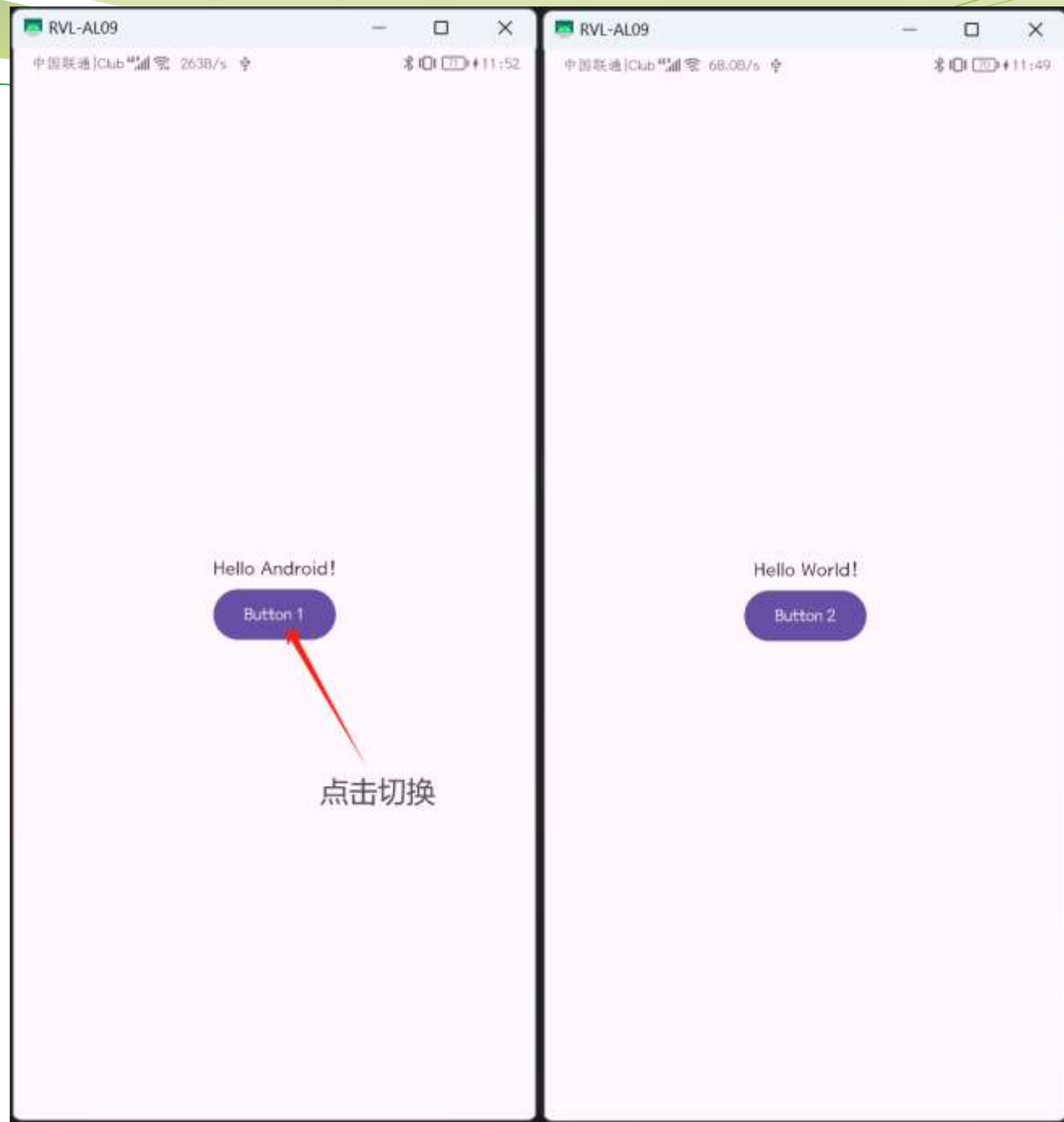


3.3 使用显式Intent

■ 明确指定启动某个Activity

Intent(Context packageContext, Class<?> cls)

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = {
            val intent = Intent(context, SecondActivity::class.java)
            activity?.startActivity(intent)
        }) {
            Text(stringResource(R.string.Button_1))
        }
    }
}
```



3.4 使用隐式Intent

- 指定更为抽象的action和category等信息
 - 由系统找出合适的Activity

```
<activity
    android:name=".SecondActivity"
    android:exported="false"
    android:label="@string/title_activity_second"
    android:theme="@style/Theme.ActivityTest">
    <intent-filter>
        <action android:name="com.example.activitytest.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = {
            val intent = Intent( action: "com.example.activitytest.ACTION_START")
            intent.`package` = context.packageName
            activity?.startActivity(intent)
        }) {
            Text(stringResource(R.string.Button_1))
        }
    }
}
```

3.4 使用隐式Intent

- 指定更为抽象的action和category等信息
 - 由系统找出合适的Activity

```
<activity
    android:name=".SecondActivity"
    android:exported="false"
    android:label="@string/title_activity_second"
    android:theme="@style/Theme.ActivityTest">
    <intent-filter>
        <action android:name="com.example.activitytest.ACTION_START" />
        <category android:name="android.intent.category.DEFAULT" />
        <category android:name="com.example.activitytest.MY_CATEGORY" />
    </intent-filter>
</activity>
```

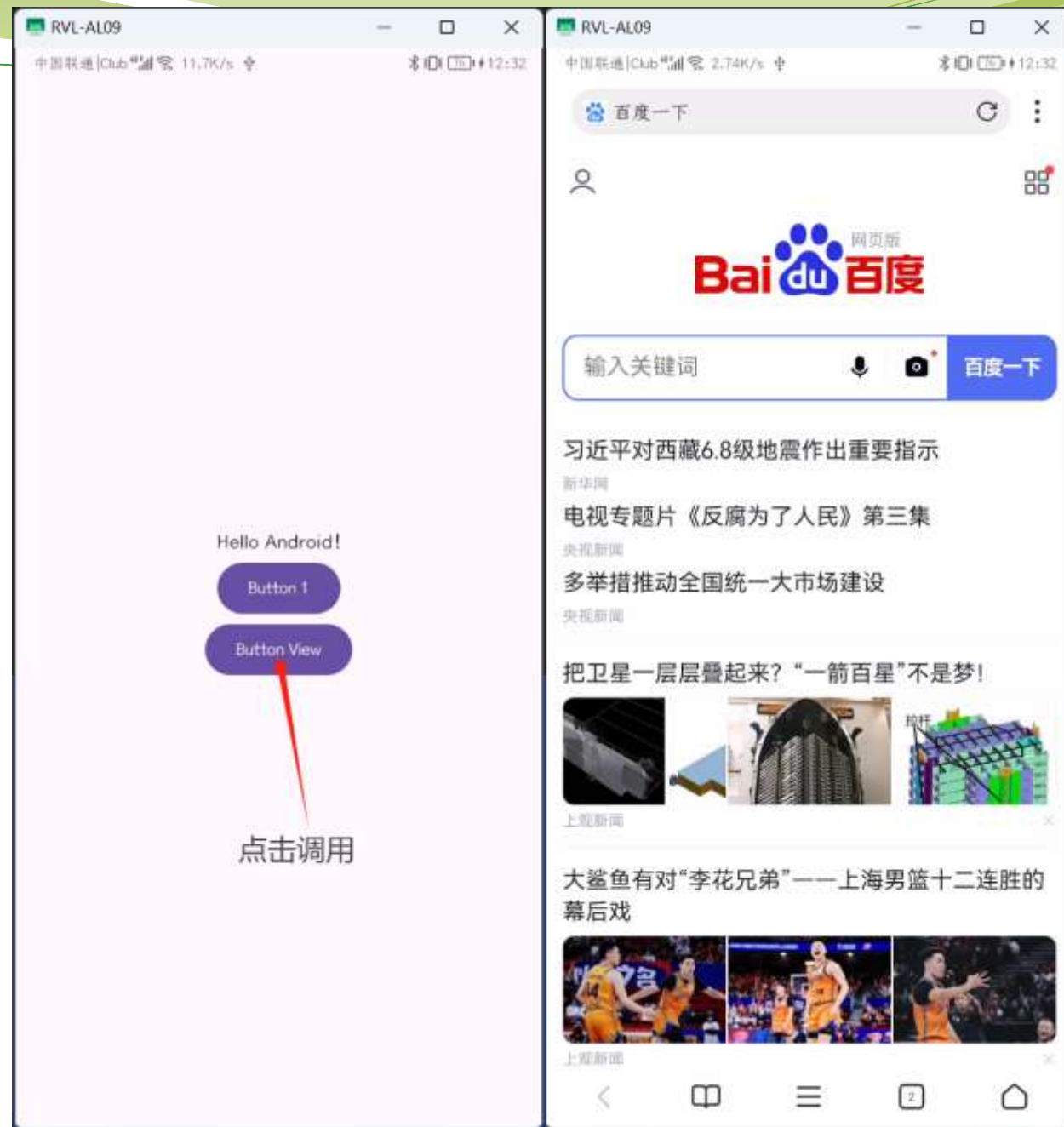
```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (
        modifier = modifier,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(
            text = "Hello $name!"
        )
        Button(onClick = {
            val intent = Intent( action: "com.example.activitytest.ACTION_START")
            intent.addCategory( category: "com.example.activitytest.MY_CATEGORY")
            intent.`package`=context.packageName
            activity?.startActivity(intent)
        }) {
            Text(stringResource(R.string.Button_1))
        }
    }
}
```


3.4 使用隐式Intent

■ 跨应用组件通信

- 调用第三方应用提供的Activity

```
Button(onClick = {  
    val intent = Intent(Intent.ACTION_VIEW)  
    intent.data = Uri.parse(uriString: "https://www.baidu.com")  
    activity?.startActivity(intent)  
}) {  
    Text(stringResource(R.string.Button_View))  
}
```



3.5 向Activity传递数据

■ 向下一个Activity传递数据:

- 重载putExtra()方法，把数据暂存在Intent当中。

```
Button(onClick = {  
    val data = "Hello SecondActivity"  
    val intent = Intent(context, SecondActivity::class.java)  
    intent.putExtra(name: "extra_data", data)  
    activity?.startActivity(intent)  
}) {  
    Text(stringResource(R.string.Button_1))  
}
```

```
Button(onClick = {  
    val extraData = activity?.intent?.getStringExtra(name: "extra_data") ?: "No Data!"  
    Toast.makeText(context, text: "Extra data is \"$extraData\"", Toast.LENGTH_LONG).show()  
}) {  
    Text(stringResource("Button Data"))  
}
```



3.5 向Activity传递数据

- 返回数据给上一个Activity:

```
Button(onClick = {  
    val intent = Intent()  
    intent.putExtra(name: "data_return", value: "Hello MainActivity")  
    activity?.setResult(RESULT_OK, intent)  
    activity?.finish()  
}) {  
    Text(stringResource(R.string.Button_2))  
}
```

3.5 向Activity传递数据

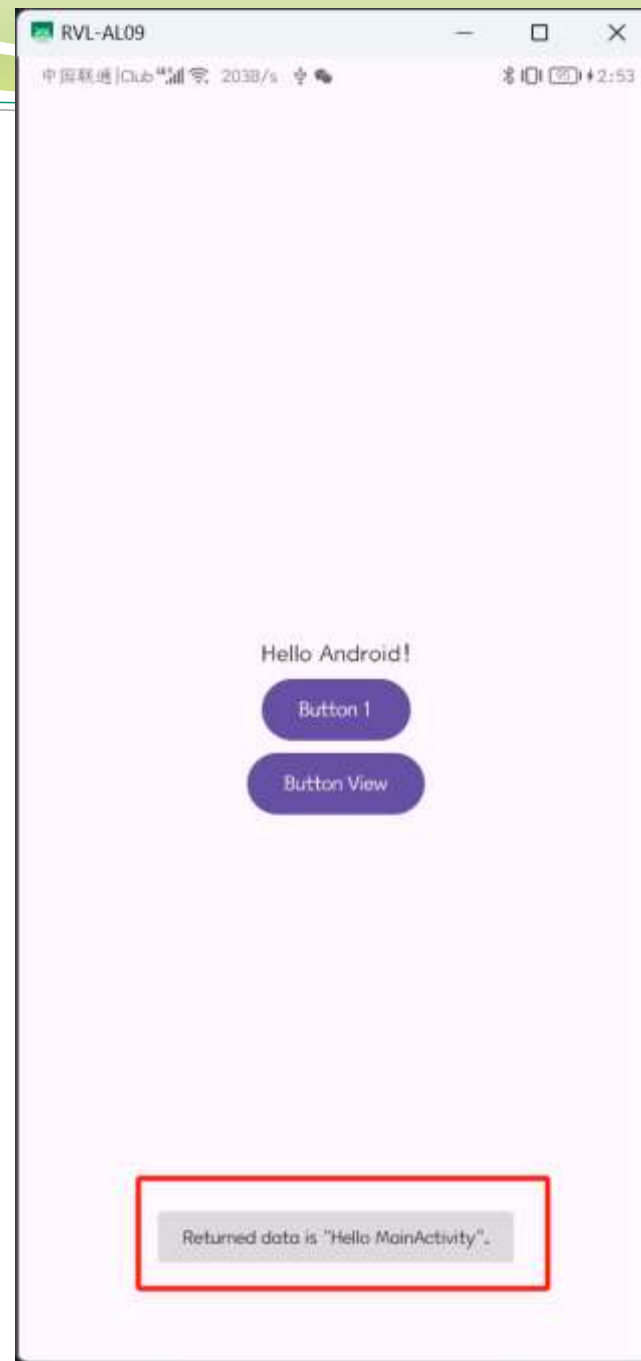
- 返回数据给上一个Activity: 使用Activity Results API

```
private val resultLauncher =
    registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
        if (result.resultCode == RESULT_OK){
            val returnData = result.data?.getStringExtra( name: "data_return")
            Toast.makeText( context: this, text: "Returned data is \"$returnData\".", Toast.LENGTH_SHORT).show()
        }
    }
```

```
@Composable
fun Greeting(name: String,
            modifier: Modifier = Modifier,
            resultLauncher: ActivityResultLauncher<Intent>?
) {
```

```
Button(onClick = {
    val data = "Hello SecondActivity"
    val intent = Intent(context, SecondActivity::class.java)
    intent.putExtra( name: "extra_data", data)
    resultLauncher?.launch(intent)
}) {
    Text(stringResource(R.string.Button_1))
}
```

3.5 向Activity传递数据



3.5 向Activity传递数据

■ 返回数据给上一个Activity: 使用Activity Results API

```
private val SecondLauncher =
    registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
        // Handle result from SecondActivity
    }
private val ThirdLauncher =
    registerForActivityResult(ActivityResultContracts.StartActivityForResult()) { result ->
        // Handle result from ThirdActivity
    }
private val activityResultLaunchers = listOf(SecondLauncher, ThirdLauncher)

.....
activityResultLaunchers.getOrNull(0)?.launch(intent)
.....
.....
activityResultLaunchers.getOrNull(1)?.launch(intent)
.....
```

4 Activity生命周期

4.1 Activity的状态

■ Activity生命周期

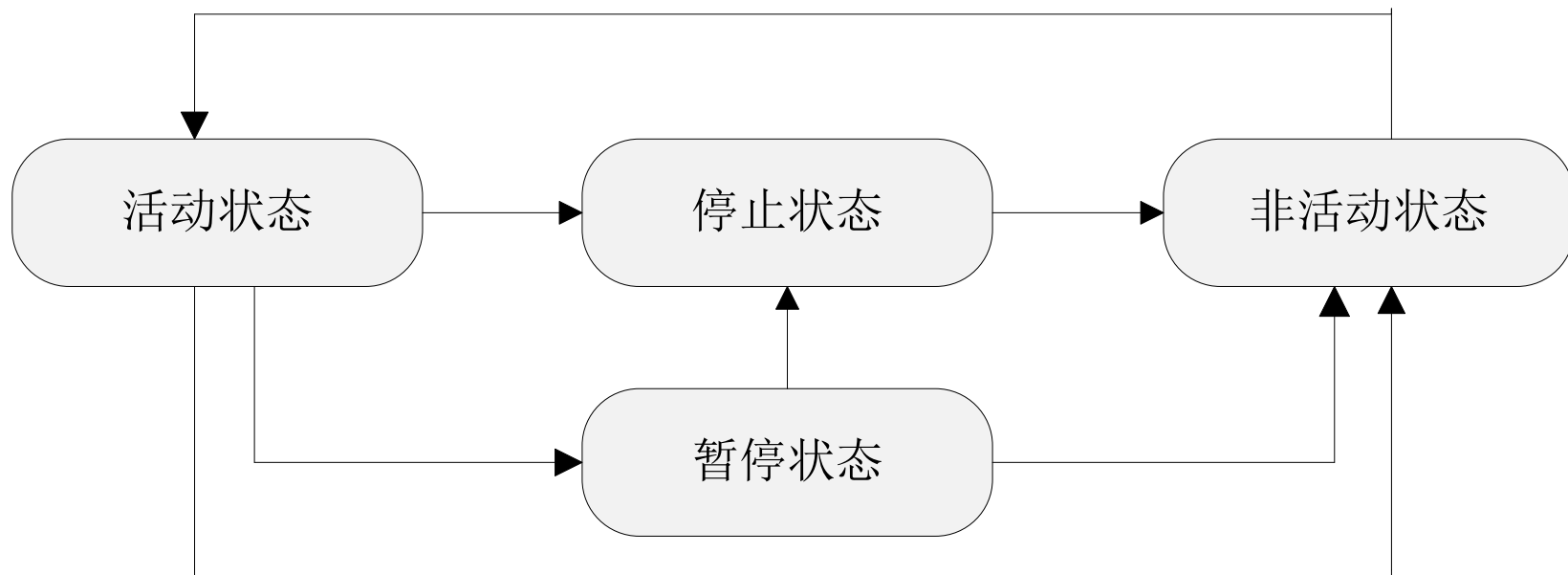
□ 指Activity从启动到销毁的过程

□ Activity表现为四种状态

- 活动状态
- 暂停状态
- 停止状态
- 非活动状态

4.1 Activity的状态

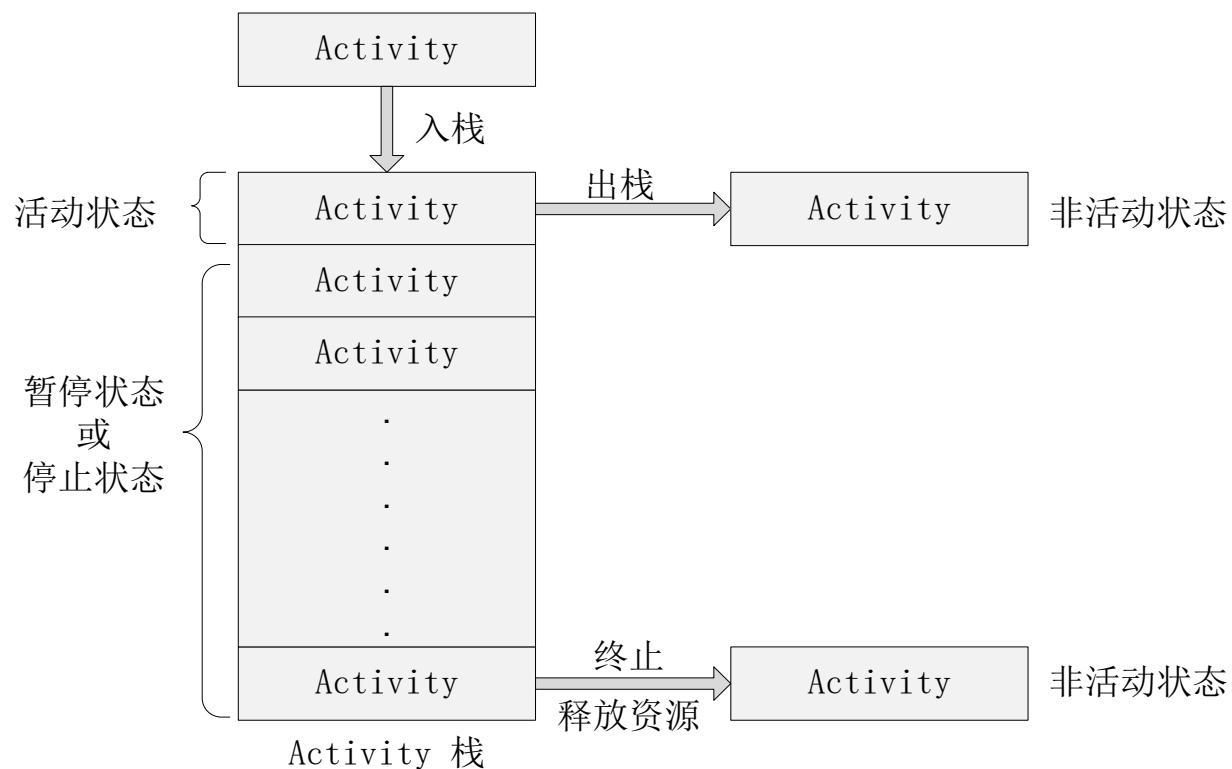
■Activity的四种状态的变换关系图



4.2 Activity返回栈

■ Activity返回栈

- 遵循"后进先出"的规则

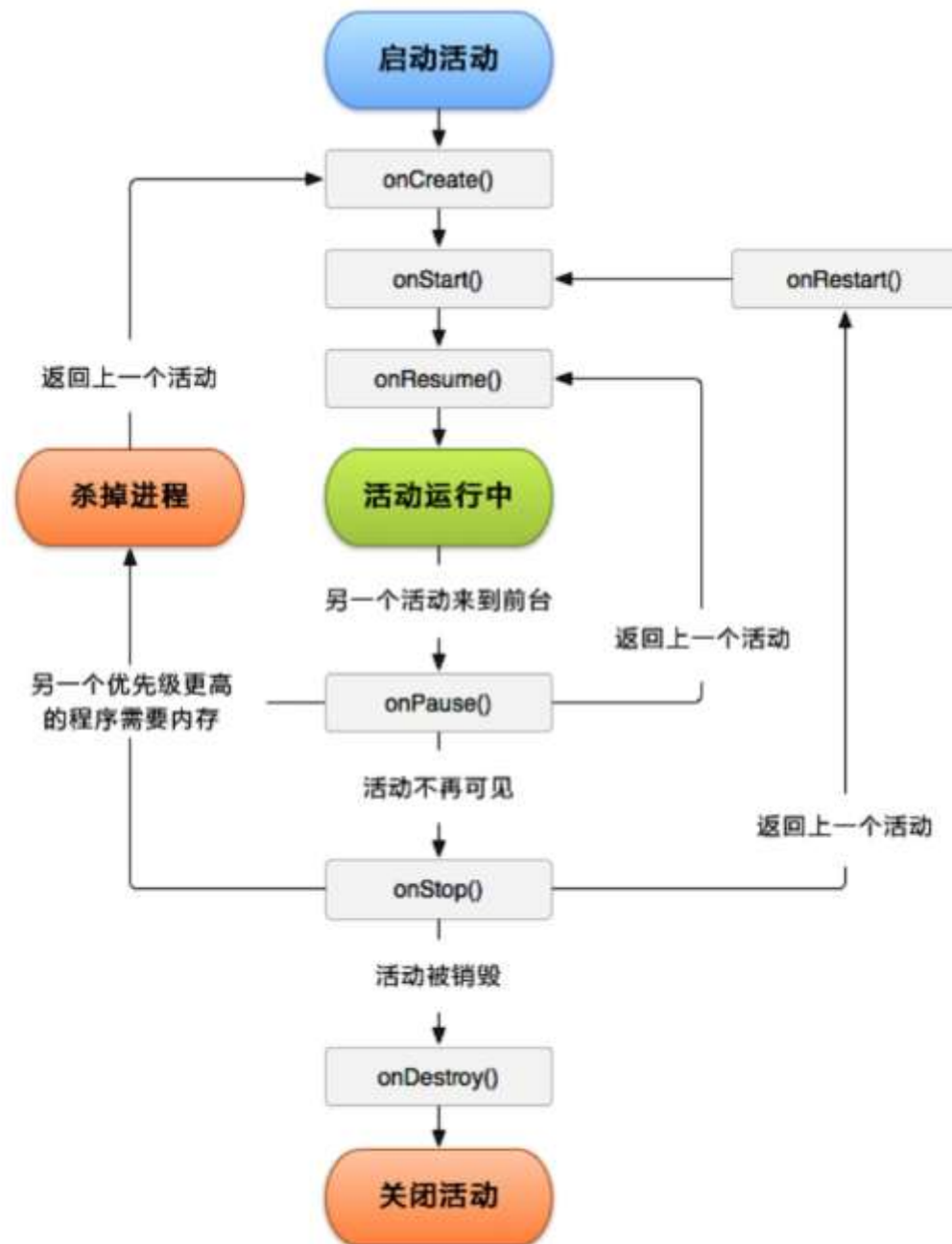


4.3 事件回调方法

- 随着Activity自身状态的变化，Android系统会调用不同的事件回调方法
- Activity的主要事件回调方法

1. `onCreate();`
2. `onStart();`
3. `onResume();`
4. `onPause();`
5. `onStop();`
6. `onDestroy();`
7. `onRestart();`

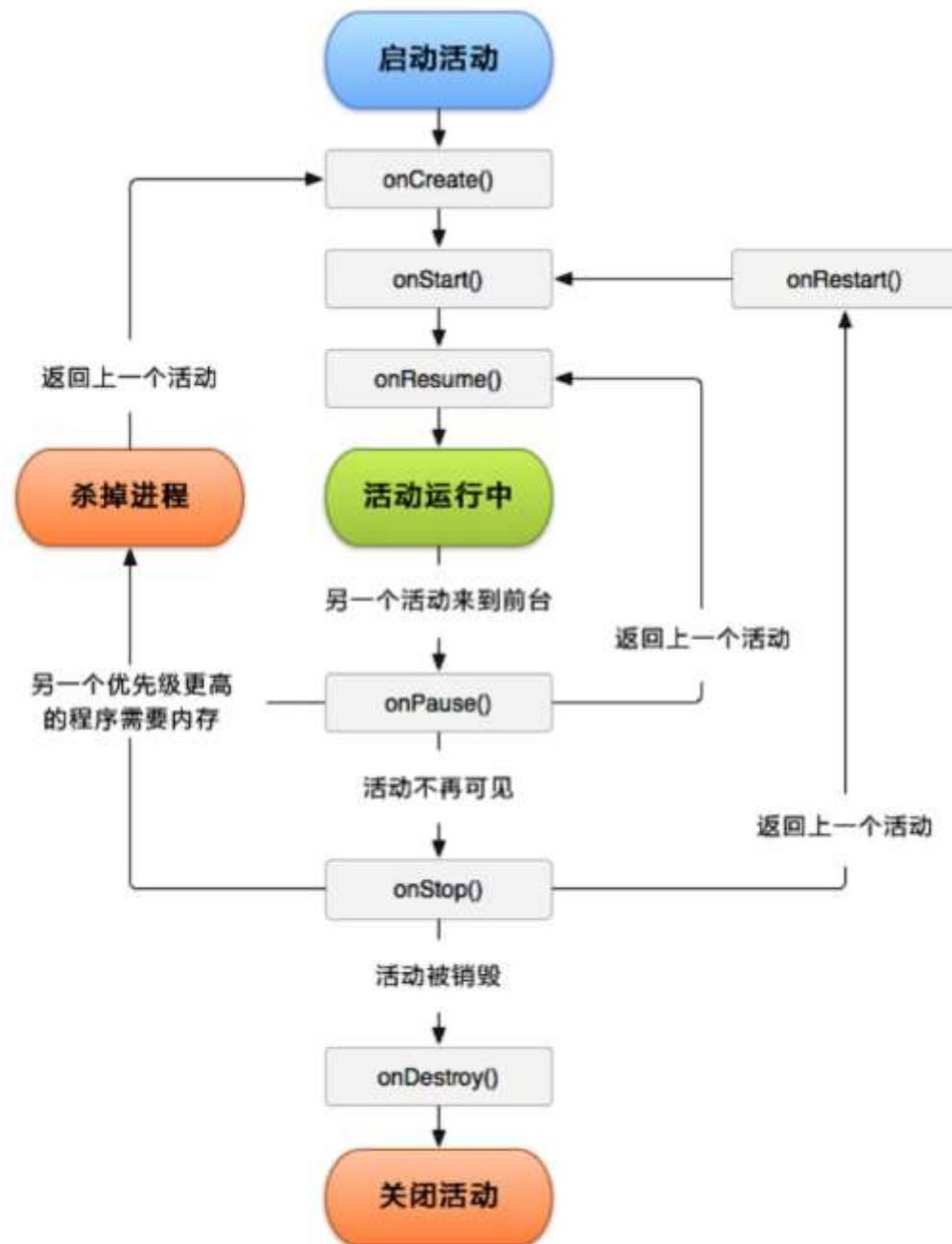
4.3 事件回调方法



4.3 事件回调方法

■ 3种生存期

- 完整生存期
- 可见生存期
- 前台生存期



4.3 事件回调方法

■ onSaveInstanceState()

- 不属于生命周期的事件回调方法，但可用于保存Activity的界面临时信息
- onSaveInstanceState()会将界面的临时信息保存在Bundle中，onCreate() 可将其恢复

函数	说明
onSaveInstanceState()	回收Activity前调用该函数，用以保存Activity的临时状态信息
onCreate()	恢复onSaveInstanceState()保存的Activity状态信息。

4.3 事件回调方法

■ onPause()和onSaveInstanceState()

- 都可用来保存界面的用户输入数据
- onPause()一般用于保存持久性数据，并将数据保存在文件系统或数据库系统中
- onSaveInstanceState()主要用来保存动态的状态信息，一般保存在Bundle中

4.4 Activity生命周期实战

```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <style name="Theme.ActivityLifeCycleTest" parent="android:Theme.Material.Light.NoActionBar" />
    <style name="Theme.ActivityLifeCycleTestDialog" parent="android:Theme.Material.Light.Dialog.NoActionBar.MinWidth" />
</resources>
```

```
<activity
    android:name=".DialogActivity"
    android:exported="false"
    android:label="DialogActivity"
    android:theme="@style/Theme.ActivityLifeCycleTestDialog" />
```

4.4 Activity生命周期实战

```
class DialogActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            ActivityLifecycleTestTheme {  
                Surface(  
                    modifier = Modifier  
                        .wrapContentSize(Alignment.Center)  
                ) {  
                    Greeting3(  
                        name = "DialogActivity"  
                    )  
                }  
            }  
        }  
    }  
}
```

```
@Composable  
fun Greeting3(name: String, modifier: Modifier = Modifier) {  
    Column(  
        modifier = modifier  
            .wrapContentSize(Alignment.Center),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.Center  
    ) {  
        Text(  
            text = "Hello $name!",  
            textAlign = TextAlign.Center,  
            modifier = Modifier  
                .fillMaxWidth()  
                .padding(vertical = 16.dp)  
        )  
    }  
}
```

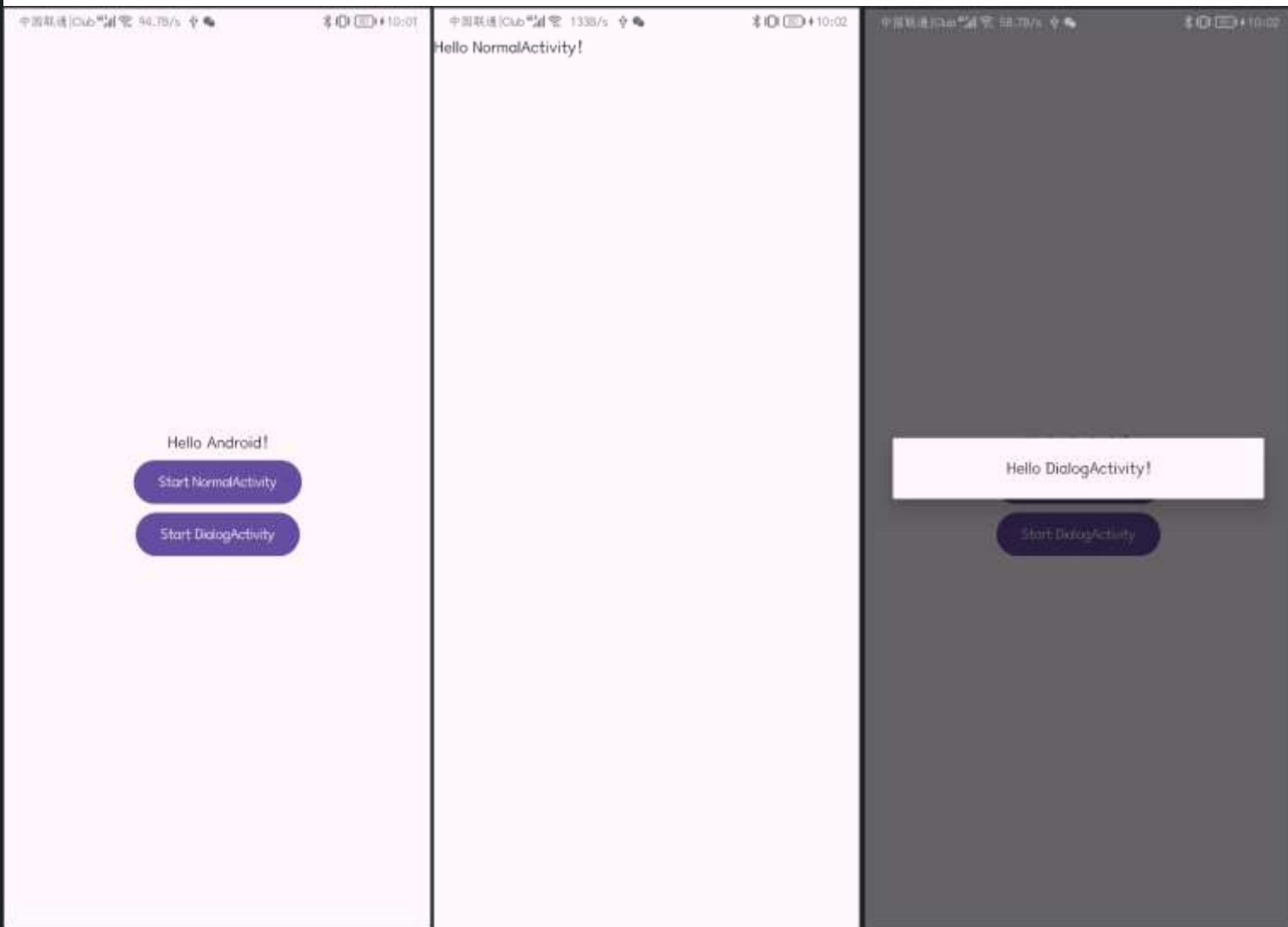

4.4 Activity生命周期实战

```
class DialogActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            ActivityLifecycleTestTheme {  
                Surface(  
                    modifier = Modifier  
                        .wrapContentSize(Alignment.Center)  
                ) {  
                    Greeting3(  
                        name = "DialogActivity"  
                    )  
                }  
            }  
        }  
    }  
}
```

```
@Composable  
fun Greeting3(name: String, modifier: Modifier = Modifier) {  
    Column(  
        modifier = modifier  
            .wrapContentSize(Alignment.Center),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.Center  
    ) {  
        Text(  
            text = "Hello $name!",  
            textAlign = TextAlign.Center,  
            modifier = Modifier  
                .fillMaxWidth()  
                .padding(vertical = 16.dp)  
        )  
    }  
}
```

4.4 Activity生命周期实战

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    val context = LocalContext.current
    val activity = context as? Activity
    Column (
        modifier = modifier
        .fillMaxSize()
        .wrapContentSize(Alignment.Center),
        horizontalAlignment = Alignment.CenterHorizontally
    ){
        Text(
            text = "Hello $name!",
            modifier = modifier
        )
        Button(
            onClick = {
                val intent = Intent(context, NormalActivity::class.java)
                activity?.startActivity(intent)
            }
        ) {
            Text(stringResource("Start NormalActivity"))
        }
        Button(
            onClick = {
                val intent = Intent(context, DialogActivity::class.java)
                activity?.startActivity(intent)
            }
        ) {
            Text(stringResource("Start DialogActivity"))
        }
    }
}
```



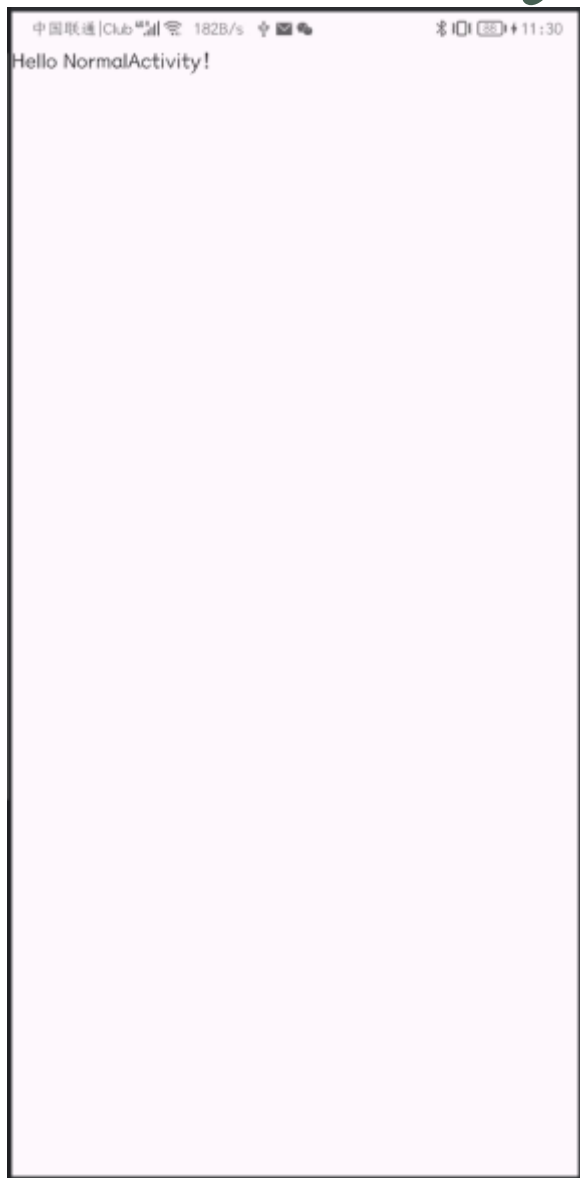
4.4 Activity生命周期实战



```
class MainActivity : ComponentActivity() {  
    private val tag = "ForMainActivity"  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        Log.d(tag, msg: "onCreate")  
        enableEdgeToEdge()  
        setContent {  
            ActivityLifecycleTestTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
    override fun onStart(){ super.onStart(); Log.d(tag, msg: "onStart") }  
    override fun onResume(){ super.onResume(); Log.d(tag, msg: "onResume") }  
    override fun onPause(){ super.onPause(); Log.d(tag, msg: "onPause") }  
    override fun onStop(){ super.onStop(); Log.d(tag, msg: "onStop") }  
    override fun onDestroy(){ super.onDestroy(); Log.d(tag, msg: "onDestroy") }  
    override fun onRestart(){ super.onRestart(); Log.d(tag, msg: "onRestart") }  
}
```

```
API 30 package:com.example.activitylifecycletest tag:ForMainActivity level:debug  
STARTED (24590) for package com.example.activitylifecycletest -----  
ForMainActivity com.example.activitylifecycletest D onCreate  
ForMainActivity com.example.activitylifecycletest D onStart  
ForMainActivity com.example.activitylifecycletest D onResume
```

4.4 Activity生命周期实战



API	package:com.example.activitylifecycletest	tag:ForMainActivity	level:debug
ForMainActivity	com.example.activitylifecycletest	D	onCreate
ForMainActivity	com.example.activitylifecycletest	D	onStart
ForMainActivity	com.example.activitylifecycletest	D	onResume
ForMainActivity	com.example.activitylifecycletest	D	onPause
ForMainActivity	com.example.activitylifecycletest	D	onStop

API	package:com.example.activitylifecycletest	tag:ForMainActivity	level:debug
ForMainActivity	com.example.activitylifecycletest	D	onPause
ForMainActivity	com.example.activitylifecycletest	D	onStop
ForMainActivity	com.example.activitylifecycletest	D	onRestart
ForMainActivity	com.example.activitylifecycletest	D	onStart
ForMainActivity	com.example.activitylifecycletest	D	onResume

4.4 Activity生命周期实战



```
API 28 package:com.example.activitylifecycletest tag:ForMainActivity level:debug
D/ForMainActivity: com.example.activitylifecycletest D onRestart
D/ForMainActivity: com.example.activitylifecycletest D onStart
D/ForMainActivity: com.example.activitylifecycletest D onResume
D/ForMainActivity: com.example.activitylifecycletest D onPause
```

```
API 28 package:com.example.activitylifecycletest tag:ForMainActivity level:debug
D/ForMainActivity: com.example.activitylifecycletest D onRestart
D/ForMainActivity: com.example.activitylifecycletest D onStart
D/ForMainActivity: com.example.activitylifecycletest D onResume
D/ForMainActivity: com.example.activitylifecycletest D onPause
D/ForMainActivity: com.example.activitylifecycletest D onResume
```

```
API 28 package:com.example.activitylifecycletest tag:ForMainActivity level:debug
D/ForMainActivity: com.example.activitylifecycletest D onPause
D/ForMainActivity: com.example.activitylifecycletest D onStop
D/ForMainActivity: com.example.activitylifecycletest D onDestroy
```

4.4 Activity生命周期实战

```
override fun onSaveInstanceState(outState: Bundle) {  
    super.onSaveInstanceState(outState)  
    val tempData = "Something you just typed"  
    outState.putString("data_key", tempData)  
}
```

override fun onCreate() 内:

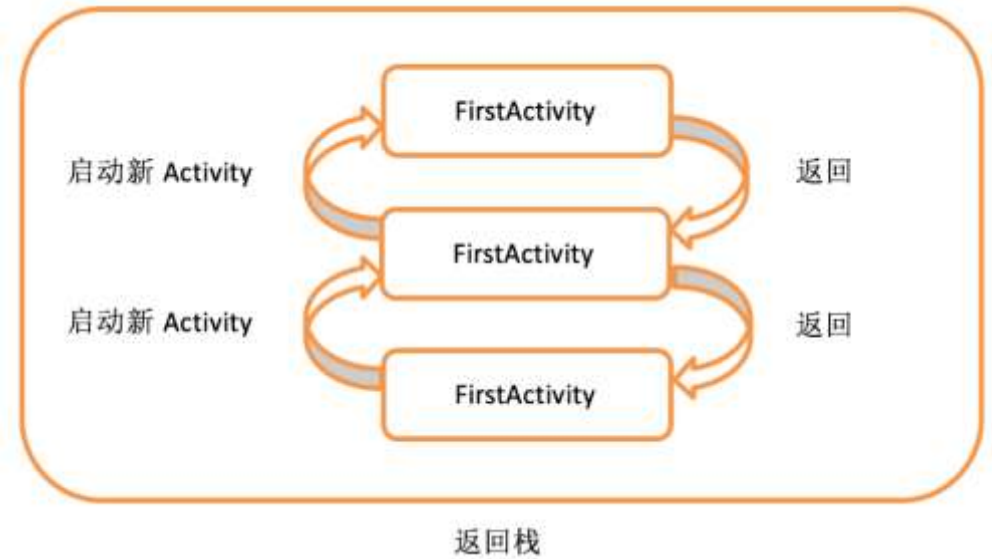
```
if (savedInstanceState != null) {  
    val tempData = savedInstanceState.getString(key: "data_key")  
    Log.d(tag, msg: "TempData is \"$tempData\".")  
}
```

5 Activity的启动模式

5.1 Activity启动模式 - standard

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        Log.d( tag: "FirstActivity", this.toString())  
    }  
}
```

```
Button(onClick = {  
    val intent = Intent(context, MainActivity::class.java)  
    activity?.startActivity(intent)  
}) {  
    Text(stringResource(R.string.button_m))  
}
```



API

package:com.example.activitymodetest level:debug tag:FirstActivity

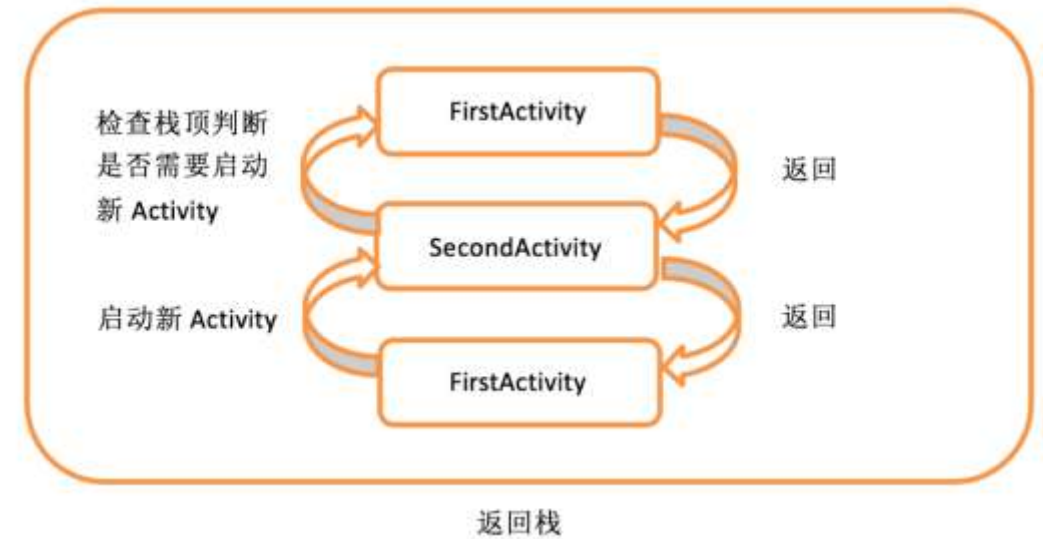
FirstActivity	com.example.activitymodetest
FirstActivity	com.example.activitymodetest
FirstActivity	com.example.activitymodetest

D	com.example.activitymodetest.MainActivity@6f0cc8b
D	com.example.activitymodetest.MainActivity@a7893bd
D	com.example.activitymodetest.MainActivity@8cecfb

5.2 Activity启动模式 - singleTop

```
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:launchMode="singleTop"
    android:label="@string/app_name"
    android:theme="@style/Theme.ActivityModeTest">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```



API ▾ package:com.example.activitymodetest level:debug tag:FirstActivity

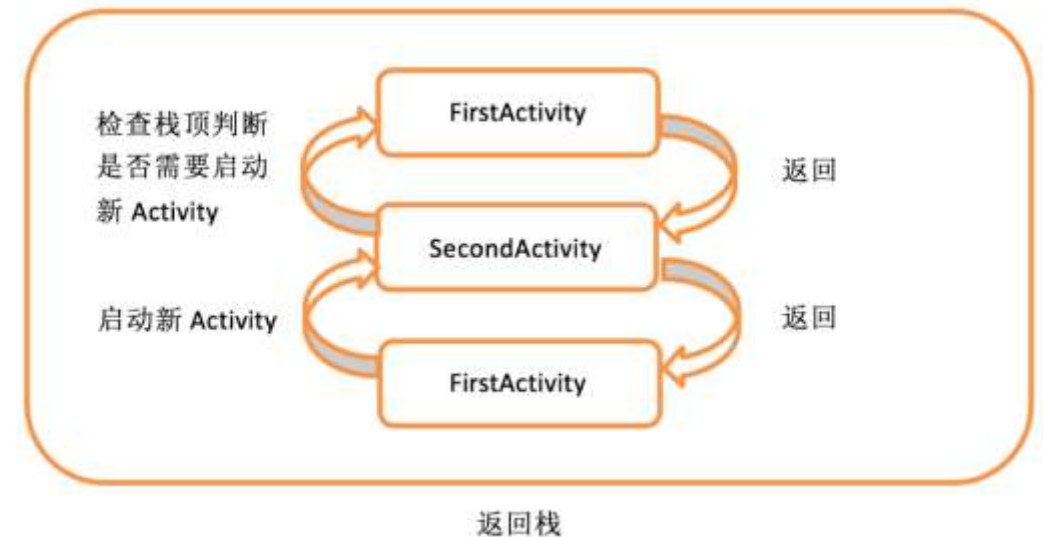
6 STARTED (10968) for package com.example.activitymodetest -----

FirstActivity com.example.activitymodetest D com.example.activitymodetest.MainActivity@6f0cc8b

5.2 Activity启动模式 - singleTop

```
class SecondActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        Log.d( tag: "SecondActivity", this.toString())  
        enableEdgeToEdge()  
    }  
}
```

```
Button(onClick = {  
    val intent = Intent(context, MainActivity::class.java)  
    activity?.startActivity(intent)  
    //activity?.finish()  
}) {  
    Text(stringResource(R.string.button_m))  
}
```



API	package:com.example.activitymodetest	level:debug	tag=:FirstActivity	tag=:SecondActivity
FirstActivity	com.example.activitymodetest		D	com.example.activitymodetest.MainActivity@15ba168
SecondActivity	com.example.activitymodetest		D	com.example.activitymodetest.SecondActivity@a7893bd
FirstActivity	com.example.activitymodetest		D	com.example.activitymodetest.MainActivity@82eeee1

5.3 Activity启动模式 - singleTask

```
android:name=".MainActivity"
android:exported="true"
android:launchMode="singleTask"
android:label="@string/app_name"
android:theme="@style/Theme.ActivityModeTest">
```

```
override fun onRestart() {
    super.onRestart(); Log.d( tag: "FirstActivity", msg: "onRestart")
}
```

```
override fun onDestroy() {
    super.onDestroy(); Log.d( tag: "SecondActivity", msg: "onDestroy")
}
```



API ▾ package:com.example.activitymodetest level:debug tag=:FirstActivity tag=:SecondActivity

```
$ STARTED (12746) for package com.example.activitymodetest -----
FirstActivity      com.example.activitymodetest    D    com.example.activitymodetest.MainActivity@6f0cc8b
SecondActivity     com.example.activitymodetest    D    com.example.activitymodetest.SecondActivity@e0c6880
FirstActivity      com.example.activitymodetest    D    onRestart
SecondActivity     com.example.activitymodetest    D    onDestroy
```

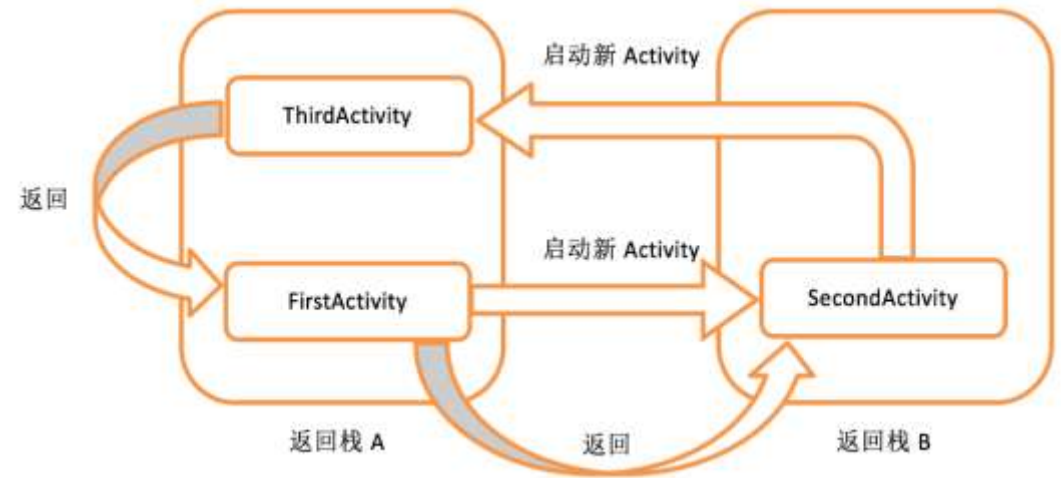

5.4 Activity启动模式 - singleInstance

```
<activity
    android:name=".SecondActivity"
    android:exported="false"
    android:label="SecondActivity"
    android:launchMode="singleInstance"
    android:theme="@style/Theme.ActivityModeTest" />
```

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Log.d("FirstActivity", this.toString())
        Log.d(tag: "FirstActivity", msg: "Task id is $taskId")
    }
}
```

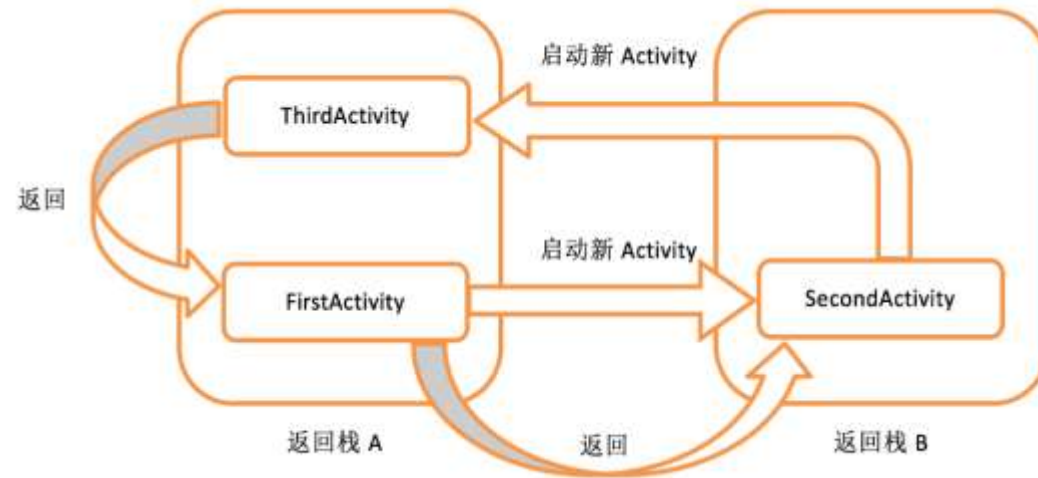
```
class SecondActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Log.d("SecondActivity", this.toString())
        Log.d(tag: "SecondActivity", msg: "Task id is $taskId")
    }
}
```

```
class ThirdActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        Log.d(tag: "ThirdActivity", msg: "Task id is $taskId")
    }
}
```



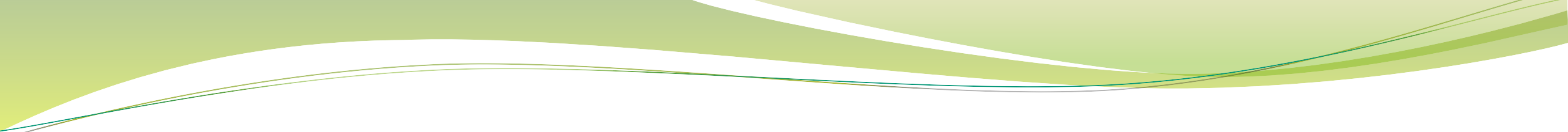
```
Button(onClick = {
    val intent = Intent(context, ThirdActivity::class.java)
    activity?.startActivity(intent)
    //activity?.finish()
}) {
    Text(stringResource(R.string.button_m))
}
```

5.4 Activity启动模式 - singleInstance



```
API  ▾ package:com.example.activitymodetest level:debug tag=:FirstActivity tag=:SecondActivity tag=:ThirdActivity

$ STARTED (15729) for package com.example.activitymodetest -----
FirstActivity      com.example.activitymodetest      D Task id is 477
SecondActivity     com.example.activitymodetest      D Task id is 478
ThirdActivity      com.example.activitymodetest      D Task id is 477
FirstActivity      com.example.activitymodetest      D onRestart
SecondActivity     com.example.activitymodetest      D onDestroy
```



感谢观看！

